EXPERIMENT-12

Implementation of echo client server using TCP/UDP sockets

Aim:

To implement an echo client server application using TCP/UDP sockets and send a message from the client to the server and receive the same message back.

Algorithm:

TCP Echo server:

- To start the TCP server
- create a server socket and wait for the client.
- Accept the client connection
- Read the message sent by the client
- Display the echoed message
- close the connection.

UDP Echo server:

- start the UDP server
- create a Datagram socket
- Receive the message from the packet
- send the same message back to the client.
- close the socket.

UDP Echo client:

- Start the UDP client.
- create a datagram socket
- Read a message from user.
- Receive the echoed packet from server
- Display the echoed message.
- close the socket.

TCP Echo client:

- start the TCP client.

· TCP Echo

Server :

```
TCP Echo server started
Received : Hello
Received : welcome
Received : bye
```

client :

```
connected to TCP Echo server
Enter message : Hello
server : Echo : Hello
server : Echo : Hello
server : Echo : bye
```

- create a socket and connect to the server
- send the message to the server.
- Display the echoed message
- close the connection.

code :

TCP Echo Server

```
import java.io.*;
import.java.net.*;
public class TCP Echo server{
 public static void main (string [] args) throws IO
                                              Expection{
  serversocket server = new serversocket (5000);
  system.out.println ("TCP Echo server started");
  socket socket = server.accept();
  Buffered Reader in = new BufferReader{
      socket.getoutput stream(), true);
string message;
 if ( message . equalsIgnored ("bye"))
 break;
}
 socket.close ()
 server.close()
 }
}
```

UDP Echo server:

```
import java.net.*;
public class UDPEcho server{
 public static void main ( string [] args) throws
                                          Exception
{
 Datagram packet packet = new Datagram socket (5000);
 system.out.println ("UDP Echo server started");
 byte[] buffer = new byte [1024];
 while (True){
   system.out.println ("Received : " + message);
   string response = "Echo: +message;
```

Output

Server:

UDP Echo Server Started

Received : Hello

Received : welcome

Received : bye

Client :

Connected to UDP Echo Server

Enter message : Hello

Server : Echo : Hello

Enter message : bye

Server : Echo : bye

```
socket . send (reply);
```

TCP Echo client :

```
import java . io . * ;
public class TCP Echo client {
  public static void main ( String[] args)
  Socket socket = new socket ("local host", 5000);
  printwriter out = new printwriter (
    socket . getOutputStream (), true);
  System . out . println (" connected to TCP Echo
                   Server");
  String message ;
   while ( true){
        system . out . print ("Enter a message");
   break ;
  }
 socket . close ();
 }
}
```

UDP Echo Server

```
import java . net . * ;
public class UDPEcho Server{
  public static void main (String[] args) throws
                                        Exception
{
  Datagram Socket socket = new Datagram Socket (5000);
  while ( true ) {
   Datagram packet packet = new Datagram packet ();
   socket . receive (packet);
   system . out . println ("Received " + message );
   string response = " Echo " + message
   byte [] data = response . getbytes ();
   socket . send ( reply);
  if (message . equalsIgnorecase ("bye"))
   break;
 }
```

UDP Echo client:

```
import java.net.*;
import java.io.*;
 public static void main (string [] args)
   system.out.println ("connected TO UDP
                           Echo server");
 while (true){
   system.out.print( "Enter message:");
   string message = keyboard.readline ();
   socket.receive (reply);
   system.out.println ("server:" + message);
    break;
 }
  socket.close ();
 }
}
```

Result:

Thus, the echo client ~~server~~ application was successfully implemented using TCP and UDP sockets and the message echoed by the server. client were successfully echo by the server.